

Python Libraries

Pandas

Pandas

About

- Python Data analysis library
- Build on top of Numpy
- Abbreviation of **P**anel **D**ata **S**ystem
- Used in production in many companies

Pandas

Tool for data Scientists

- Managing data
- Cleaning data
- Analyzing
- Modeling data
- Organizing the data in a form suitable

Pandas

There are two data structure

- Series

- Series is a one-dimensional array-like object containing an array of data

- Dataframe

- Dataframe is a data structure containing and ordered collections of columns
 - Each column may hold numeric, string, Boolean etc,
 - Dataframe has both row and column index

Pandas

Series

■ Create a series

```
import pandas as pd
obj = pd.Series([1, 5, 6, 8, 3])
```

```
0    1
1    5
2    6
3    8
4    3
dtype: int64
```

■ Adding index to series

```
import pandas as pd
obj = pd.Series([1, 5, 6, 8, 3], index=['a', 'b', 'c', 'd', 'e'])
```

```
a    1
b    5
c    6
d    8
e    3
dtype: int64
```

Pandas

DataFrame

- A pandas DataFrame can be created using various input like
 - Lists
 - Dict
 - Series
 - Numpy ndarrays
 - Another DataFrame

Pandas

DataFrame

- Create an Empty DataFrame

```
import pandas as pd
df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

- Create a DF from List

```
import pandas as pd
data = [1,6,7,5,5]
df = pd.DataFrame(data=data)
print(df)
```

```
0
0  1
1  6
2  7
3  5
4  5
```

Pandas

DataFrame

- Create a DF from List

```
import pandas as pd
data = [['A',6],['B',5],['C',3]]
df = pd.DataFrame(data=data, columns=['col1', 'col2'], dtype=float)
print(df)
```

	col1	col2
0	A	6.0
1	B	5.0
2	C	3.0

Pandas

DataFrame

- Create a DF from Dict of ndarrays/Lists

```
import pandas as pd
data = {'Name': ['A', 'B'], 'Age': ['25', '30'], 'City': ['HN', 'HCM']}
df = pd.DataFrame(data=data, dtype=float)
print(df)
```

	Name	Age	City
0	A	25.0	HN
1	B	30.0	HCM

Pandas

DataFrame

- Create a DF from List of Dicts

```
import pandas as pd
data = [{'A':1, 'B':2}, {'A':2, 'B':3, 'C':5}]
df = pd.DataFrame(data=data, dtype=float)
print(df)
```

	A	B	C
0	1.0	2.0	NaN
1	2.0	3.0	5.0

Pandas

DataFrame

- Create a DF from Dict of Series

```
import pandas as pd
data = {'col1': pd.Series([1,2,3], index=['a', 'b', 'c']),
        'col2': pd.Series([4,5,6,7], index=['a','b','c','d'])}
df = pd.DataFrame(data=data, dtype=float)
print(df)
```

	col1	col2
a	1.0	4.0
b	2.0	5.0
c	3.0	6.0
d	NaN	7.0

Pandas

Read DF from csv file

```
df = pd.read_csv('/content/pokemon_data.csv')
```

```
df.head()
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False

Pandas

Read DF from excel and text files

From Excel files

```
df_excel = pd.read_excel('/content/pokemon_data.xlsx')
```

From text files

```
df_txt = pd.read_csv('/content/pokemon_data.txt', delimiter='\t')
```

Pandas

- Read only a few first rows
 - `df.head(n=5)`
- Read only a few last rows
 - `df.tail(n=5)`
- Read a few random rows
 - `df.sample(n=5)`

Pandas

- Get all columns name
 - `df.columns`
- Read a specific columns
 - `df['column_name']`
- Read unique values of a columns
 - `df['column_name'].unique()`

Pandas

- Read some columns
 - `df['col1', 'col2', 'col3']`
- Read a specific row
 - `df.iloc[1]`
- Read a few rows
 - `df.iloc[1:4]`

Pandas

Information of data

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   #               800 non-null   int64
1   Name            800 non-null   object
2   Type 1          800 non-null   object
3   Type 2          414 non-null   object
4   HP              800 non-null   int64
5   Attack          800 non-null   int64
6   Defense         800 non-null   int64
7   Sp. Atk         800 non-null   int64
8   Sp. Def         800 non-null   int64
9   Speed           800 non-null   int64
10  Generation      800 non-null   int64
11  Legendary       800 non-null   bool
12  Total           800 non-null   int64
dtypes: bool(1), int64(9), object(3)
memory usage: 75.9+ KB
```

Pandas

Describe data

```
df.describe()
```

	#	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Total
count	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000
mean	362.813750	69.258750	79.001250	73.842500	72.820000	71.902500	68.277500	3.32375	435.10250
std	208.343798	25.534669	32.457366	31.183501	32.722294	27.828916	29.060474	1.66129	119.96304
min	1.000000	1.000000	5.000000	5.000000	10.000000	20.000000	5.000000	1.00000	180.00000
25%	184.750000	50.000000	55.000000	50.000000	49.750000	50.000000	45.000000	2.00000	330.00000
50%	364.500000	65.000000	75.000000	70.000000	65.000000	70.000000	65.000000	3.00000	450.00000
75%	539.250000	80.000000	100.000000	90.000000	95.000000	90.000000	90.000000	5.00000	515.00000
max	721.000000	255.000000	190.000000	230.000000	194.000000	230.000000	180.000000	6.00000	780.00000

Pandas

Filter columns by condition

```
df[df['Type 1'] == 'Grass'].head()
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	318
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	405
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
48	43	Oddish	Grass	Poison	45	50	55	75	65	30	1	False	320

Pandas

Filter columns by multiple conditions

```
df[(df['Type 1'] == 'Grass') & (df['Type 2'] == 'Poison') & (df['HP'] > 50)].head()
```

#		Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	405
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
49	44	Gloom	Grass	Poison	60	65	70	85	75	40	1	False	395
50	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	490

Pandas

Filter rows that contain a specific value

```
df[df['Name'].str.contains('Mega')].head()
```

#		Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
7	6	CharizardMega Charizard X	Fire	Dragon	78	130	111	130	85	100	1	False	634
8	6	CharizardMega Charizard Y	Fire	Flying	78	104	78	159	115	100	1	False	634
12	9	BlastoiseMega Blastoise	Water	NaN	79	103	120	135	115	78	1	False	630
19	15	BeedrillMega Beedrill	Bug	Poison	65	150	40	15	80	145	1	False	495

Pandas

■ Aggregate Statistics (Groupby)

- Count
- Sum
- Mean

```
df.groupby(by='Type 1').count().head()
```

	#	Name	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
Type 1												
Bug	69	69	52	69	69	69	69	69	69	69	69	69
Dark	31	31	21	31	31	31	31	31	31	31	31	31
Dragon	32	32	21	32	32	32	32	32	32	32	32	32
Electric	44	44	17	44	44	44	44	44	44	44	44	44
Fairy	17	17	2	17	17	17	17	17	17	17	17	17

```
df.groupby(by=['Name']).sum().head()
```

	#	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
Name										
Abomasnow	460	90	92	75	92	85	60	4	0	494
AbomasnowMega Abomasnow	460	90	132	105	132	105	30	4	0	594
Abra	63	25	20	15	105	55	90	1	0	310
Absol	359	65	130	60	75	60	75	3	0	465
AbsolMega Absol	359	65	150	60	115	60	115	3	0	565

```
df.groupby(by=['Type 1', 'Type 2']).mean().head()
```

		#	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
Type 1	Type 2										
Bug	Electric	595.500000	60.0	62.000000	55.000000	77.000000	55.000000	86.500000	5.000000	0.0	395.5
	Fighting	214.000000	80.0	155.000000	95.000000	40.000000	100.000000	80.000000	2.000000	0.0	550.0
	Fire	636.500000	70.0	72.500000	60.000000	92.500000	80.000000	80.000000	5.000000	0.0	455.0
	Flying	286.285714	63.0	70.142857	61.571429	72.857143	69.071429	82.857143	2.857143	0.0	419.5
	Ghost	292.000000	1.0	90.000000	45.000000	30.000000	30.000000	40.000000	3.000000	0.0	236.0

Pandas

- Define the aggregation procedure outside of the groupby operation

```
# Define the aggregation procedure outside of the groupby operation
aggregations = {
    'Attack': 'sum',
    'Defense': 'count'
}
df.groupby('Name').agg(aggregations).head()
```

	Attack	Defense
Name		
Abomasnow	92	1
AbomasnowMega Abomasnow	132	1
Abra	20	1
Absol	130	1
AbsolMega Absol	150	1

Pandas

■ Apply function for column

- Parse datetime
- Transform data
- `df.apply(<function>)`

```
import pandas as pd
import dateutil

# Load data from csv file
data = pd.DataFrame.from_csv('phone_data.csv')
# Convert date from string to date times
data['date'] = data['date'].apply(dateutil.parser.parse,
dayfirst=True)
```


Pandas

- **Pandas with SQL**

- Read data from connection to DB
- Execute query to DB

Pandas

■ Pandas with SQL

- Create a database connect to a SQL database

```
import sqlite3
import pandas as pd
```

```
def create_connection(db_file):
    """ create a database connection to a SQLite database """
    conn = None
    try:
        conn = sqlite3.connect(db_file)
        print(sqlite3.version)
        return conn
    except Error as e:
        print(e)
```

```
database = r'Events.db'
```

```
# create a database connection
conn = create_connection(database)
```

Pandas

■ Pandas with SQL

■ Execute query

```
sql_customerData = "SELECT * FROM Event"  
  
df = pd.read_sql(sql_customerData, conn)  
df.head()
```

	detailDescription	eventID	startTime		name	endTime	creatDate	location
0	Học Python	1	2020-09-09	Using database with python	2020-10-09	2020-05-06		1
1	Học SQL	2	2020-06-08		SQL	2020-10-06	2020-05-06	2
2	Học C#	3	2020-09-03		C#	2020-11-19	2020-08-03	3

Pandas

Pandas Practice

Thank you